

Universitatea POLITEHNICA București

Facultatea de Automatică și Calculatoare

Agent inteligent pentru jocul Block Blast utilizând învățare prin recompensă

Documentație proiect

Autor: Potop Horia-Ioan

Grupa: 333AA

An: 2026

Cuprins

1	Introducere	3
2	Context teoretic și realizări similare	3
2.1	Învățarea prin recompensă	3
2.2	Algoritmul PPO	4
2.3	Action masking	4
2.4	Reprezentări CNN pentru jocuri	4
2.5	Legătura cu multimedia și reprezentarea vizuală	4
2.6	Realizări similare	5
3	Analiza problemei	5
3.1	Regulile jocului	5
3.2	Spațiul de acțiuni	5
3.3	Dificultăți	5
4	Arhitectura aplicației	6
5	Modele neuronale, alternative testate și funcții de recompensă	7
5.1	De ce nu a fost folosită o metodă tabelară clasică	7
5.2	MLP inițial	8
5.3	Rețele CNN folosite	8
5.4	Funcții de recompensă	8
6	Generatorul de piese	9
7	Workflow tehnic complet al antrenării	9
7.1	Pornirea unei rulări de antrenare	9
7.2	Observația primită de model	10
7.3	Calculul măștii de acțiuni	10
7.4	Trecerea prin CNN	10
7.5	Actorul, criticul și acțiunea aleasă	12
7.6	Aplicarea acțiunii în mediu	12
7.7	Colectarea rollout-ului	12
7.8	Actualizarea PPO	12
7.9	Monitorizarea în TensorBoard	13
8	Euristica de referință	13
9	Experimente și rezultate	14
9.1	Metodologia experimentală	14
9.2	Familii de experimente încercate	14
9.3	Rezultate reprezentative	16

9.4	Observații despre experimentul 4x4	16
9.5	Grafice și capturi pentru rezultate	16
10	Testare și validare	18
10.1	Testare în timpul antrenării	18
10.2	Testare după antrenare în watch.py	19
11	Concluzii	19
12	Utilizarea instrumentelor LLM	20

1 Introducere

Proiectul prezintă implementarea și evaluarea unui agent inteligent capabil să joace un joc de tip *Block Blast*, folosind metode de învățare prin recompensă. Jocul poate fi privit ca o problemă secvențială de decizie: la fiecare pas agentul primește o tablă, trei piese disponibile și trebuie să aleagă piesa și poziția de plasare. Obiectivul este maximizarea duratei de supraviețuire și a numărului de linii eliminate.

Tema este relevantă deoarece jocurile de tip puzzle oferă un cadru compact pentru studierea unor concepte importante din învățarea automată: explorare, recompense întârziate, reprezentări spațiale, politici stocastice și generalizare pe stări noi. Spre deosebire de jocuri cu reguli complet determinate și spații de acțiuni reduse, în *Block Blast* apar frecvent blocaje cauzate de geometria pieselor și de modul în care acestea ocupă tabla. Din acest motiv, proiectul nu se rezumă la antrenarea unui model, ci include proiectarea mediului, a generatorului de piese, a funcțiilor de recompensă, a arhitecturilor neuronale și a procedurilor de evaluare.

Obiectivele principale ale proiectului sunt:

- implementarea unui mediu de joc compatibil cu interfața *Gymnasium*;
- definirea reprezentării stării, a acțiunilor valide și a mecanismului de *action masking*;
- antrenarea unor agenți PPO cu extractoare de caracteristici CNN;
- proiectarea și compararea mai multor funcții de recompensă;
- construirea unei euristici de referință pentru validarea generatorului;
- realizarea unei interfețe vizuale pentru testarea manuală, automată și comparativă;
- evaluarea variantei principale 8x8 și testarea suplimentară a unei variante 4x4.

Contribuția personală constă în implementarea completă a logicii jocului, a mediului de antrenare, a mai multor variante de generatoare de piese, a arhitecturilor CNN folosite de PPO, a funcțiilor de recompensă și a instrumentelor de evaluare. De asemenea, au fost realizate numeroase experimente pentru a identifica limitele abordării și configurațiile promițătoare.

2 Context teoretic și realizări similare

2.1 Învățarea prin recompensă

Învățarea prin recompensă, cunoscută în literatura de specialitate ca *Reinforcement Learning*, studiază modul în care un agent poate învăța să ia decizii prin interacțiune repetată cu un mediu. Formal, problema este modelată ca un proces decizional Markov, în care la momentul t agentul observă o stare s_t , alege o acțiune a_t , primește o recompensă r_t și ajunge într-o stare nouă s_{t+1} [1]. Scopul este maximizarea recompensei cumulate:

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}, \quad (1)$$

unde $\gamma \in [0, 1]$ este factorul de discount. În proiect, valorile mai mici ale lui γ au fost investigate deoarece decizia locală de plasare a unei piese are un efect imediat puternic, iar un discount prea mare poate face semnalul de învățare mai dificil de stabilizat.

2.2 Algoritmul PPO

Pentru antrenare a fost folosit algoritmul Proximal Policy Optimization, introdus de Schulman et al. [2]. PPO optimizează o politică stocastică, dar limitează modificările prea mari între politica veche și cea nouă. Intuiția este că agentul trebuie să învețe progresiv, fără ca actualizările să distrugă complet comportamentul deja dobândit. Funcția obiectiv folosește un termen de tip *clipped surrogate objective*:

$$L^{CLIP}(\theta) = \mathbb{E}_t [\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon)\hat{A}_t)], \quad (2)$$

unde $r_t(\theta)$ este raportul probabilităților dintre politica nouă și cea veche, iar $\hat{A}_t$ este estimarea avantajului.

2.3 Action masking

În jocul Block Blast, nu toate acțiunile sunt valide în fiecare stare. O piesă poate depăși tabla sau se poate suprapune peste celule deja ocupate. În loc ca agentul să fie penalizat după alegerea unei acțiuni imposibile, a fost folosit *action masking*: acțiunile invalide primesc probabilitate zero înainte de eșantionarea acțiunii. Implementarea folosește extensia MaskablePPO din `sb3-contrib` [3].

2.4 Reprezentări CNN pentru jocuri

Rețelele convoluționale sunt potrivite pentru date cu structură spațială, precum imagini sau table de joc. Ele au fost dezvoltate pentru a exploata localitatea și partajarea ponderilor în date bidimensionale [4], iar în învățarea profundă modernă sunt o alegere standard pentru reprezentări vizuale și spațiale [5]. În acest proiect, tabla de joc este reprezentată ca o matrice binară, iar hărțile acțiunilor valide sunt tratate ca alte canale spațiale. Ideea este asemănătoare cu modul în care agenții de jocuri pot primi drept intrare reprezentări bidimensionale ale mediului [6]. Diferența importantă este că aici nu se folosește o imagine RGB, ci o reprezentare discretă explicită: celule ocupate, piese disponibile și poziții legale.

2.5 Legătura cu multimedia și reprezentarea vizuală

Deși proiectul nu procesează imagini naturale, sunet sau video, el are o legătură directă cu multimedia prin modul de reprezentare a informației vizuale. Tabla de joc este tratată ca o imagine binară de dimensiune mică, în care fiecare celulă funcționează ca un pixel semantic: 0 înseamnă spațiu liber, iar 1 înseamnă spațiu ocupat. Peste această reprezentare se adaugă hărți suplimentare pentru acțiuni valide, asemănătoare unor canale auxiliare din procesarea imaginilor.

Prin urmare, CNN-ul folosește principii similare cu cele din înțelegerea imaginilor: filtre locale, detectarea de vecinătăți, combinarea canalelor și extragerea de tipare spațiale. Diferența este că imaginea nu provine de la o cameră, ci dintr-un grid generat de motorul jocului. Această reprezentare reduce dimensiunea problemei față de o imagine reală, dar păstrează complexitatea geometrică: linii aproape complete, spații compacte, goluri, margini și forme care se potrivesc

în anumite regiuni. Interfața `watch.py` completează partea multimedia prin vizualizarea tablei, a pieselor și a deciziilor agentului, ceea ce permite testarea calitativă a comportamentului.

2.6 Realizări similare

Există și alte încercări de a construi agenți pentru jocuri de tip Block Blast, de exemplu *BlockBlast-Game-AI-Agent* și *BlockBlastAI* [7, 8]. Aceste proiecte confirmă că PPO, DQN, PyTorch, Gymnasium și Pygame sunt alegeri naturale pentru problemă, dar sunt orientate mai mult spre obținerea rapidă a unui agent funcțional. În general, abordările publice folosesc generatoare mai permissive, dificultate mai puțin adaptivă și mai puține experimente pe recompense, arhitecturi și generalizare. Diferența proiectului de față este componenta experimentală: au fost testate mai multe familii de recompense, valori pentru γ , rate de învățare, variante CNN, seed-uri, generatoare adaptive și o euristică de referință.

3 Analiza problemei

3.1 Regulile jocului

Tabla este o grilă de dimensiune $N \times N$. În varianta inițială, $N = 8$, iar ulterior a fost implementată și o variantă $N = 4$ pentru a accelera experimentele și pentru a reduce complexitatea decizională. La fiecare rundă sunt disponibile trei piese. Agentul alege una dintre piesele disponibile și o poziție de ancorare pe tablă. Dacă plasarea este validă, piesa ocupă celulele corespunzătoare. După plasare, toate liniile și coloanele complet ocupate sunt eliminate.

Un episod se termină atunci când niciuna dintre piesele rămase nu mai poate fi plasată. Scorul relevant pentru proiect nu este doar numărul de linii eliminate, ci și numărul de etape supraviețuite și calitatea poziționării pieselor.

3.2 Spațiul de acțiuni

Acțiunea este codificată prin alegerea unei piese din setul curent și a unei poziții pe tablă. Pentru trei piese și o tablă $N \times N$, numărul maxim de acțiuni este:

$$|\mathcal{A}| = 3 \cdot N \cdot N. \quad (3)$$

Astfel, pentru varianta 8x8 există $3 \cdot 8 \cdot 8 = 192$ acțiuni, iar pentru varianta 4x4 există $3 \cdot 4 \cdot 4 = 48$ acțiuni. În practică, doar o parte dintre aceste acțiuni sunt valide în fiecare stare.

3.3 Dificultăți

Problema este dificilă din mai multe motive:

- recompensa pentru o plasare bună poate apărea abia după mai multe mutări;
- piesele mari pot bloca rapid tabla dacă sunt plasate greșit;

permis revenirea la versiuni intermediare ale modelelor și compararea performanței în diferite momente ale antrenării.

Tabela 1: Rolul principal al fișierelor din proiect.

Fișier	Rol	Observații
<code>blocks.py</code>	definește piesele și categoriile lor	folosit de generator și de interfața vizuală
<code>engine.py</code>	implementează regulile jocului	plasare, linii, final de joc, generator
<code>env.py</code>	expune jocul ca mediu de antrenare	observații, recompense, action mask, CNN-uri
<code>train.py</code>	pornește antrenarea PPO	hiperparametri, checkpoint-uri, TensorBoard
<code>watch.py</code>	permite testarea vizuală	model, euristică, om vs AI, solver/editor

Separarea aceasta a fost păstrată intenționat pe tot parcursul proiectului. `engine.py` rămâne responsabil doar de reguli, astfel încât aceeași logică poate fi folosită atât la antrenare, cât și în interfața vizuală. `env.py` adaugă peste motor noțiunile necesare pentru învățare prin recompensă: observație, acțiune, mască și recompensă. `train.py` nu modifică regulile jocului, ci controlează doar modul în care PPO interacționează cu mediul. Din acest motiv, Tabelul 2 poate fi privit ca punctul de legătură dintre arhitectura aplicației și arhitectura modelului.

Observația returnată agentului este un dicționar cu patru componente:

Tabela 2: Reprezentarea observației pentru variantele 8x8 și 4x4.

Componentă	8x8	4x4
board	(1, 8, 8)	(1, 4, 4)
valid_actions	(3, 8, 8)	(3, 4, 4)
hand	(3, 3, 3)	(3, 3, 3)
available	(3)	(3)
Spațiu acțiuni	192	48

5 Modele neuronale, alternative testate și funcții de recompensă

5.1 De ce nu a fost folosită o metodă tabelară clasică

O variantă clasică de tip Q-learning tabelar nu este potrivită aici, deoarece tabla 8x8 are până la 2^{64} configurații posibile înainte de a include piesele disponibile. Un tabel explicit ar fi prea mare, iar multe stări ar apărea rar. De aceea, proiectul folosește rețele neuronale care nu memorează stările, ci învață tipare generale: zone compacte, linii aproape complete și spații care rămân jucabile după mutare.

5.2 MLP inițial

În etapele inițiale a fost luată în considerare și o abordare cu MLP, adică o rețea complet conectată care primește starea aplatizată. Este rapidă și simplă, dar pierde structura spațială a tablei. Pentru Block Blast, această structură este esențială: liniile, coloanele, colțurile, golurile și contactul dintre forme sunt relații geometrice, motiv pentru care MLP-ul a rămas doar o variantă de bază.

5.3 Rețele CNN folosite

CNN-ul a fost ales deoarece tabla și hărțile de acțiuni valide au structură bidimensională. În model, tabla ocupată și cele trei hărți de acțiuni valide sunt concatenate ca patru canale spațiale, iar piesele disponibile sunt procesate separat prin straturi liniare. Caracteristicile rezultate sunt combinate și trimise către PPO, unde actorul alege acțiunea, iar criticul estimează valoarea stării.

Au fost încercate trei variante. **CNN de bază** a fost modelul principal: suficient de expresiv pentru tipare spațiale, dar încă rapid. **CNN V2** a adăugat blocuri mai adânci și normalizare, însă costul mai mare nu a garantat rezultate mai bune. **Action-aware CNN** a procesat separat fiecare piesă și harta ei de acțiuni valide, fiind mai apropiat de o evaluare euristică, dar a fost mult mai lent pe CPU.

5.4 Funcții de recompensă

Au fost testate mai multe variante de recompensă. Forma generală a recompensei este:

$$R = R_{\text{linie}} + R_{\text{contact}} + R_{\text{etapa}} - P_{\text{final}} - P_{\text{gauri}}. \quad (4)$$

În experimentele finale, termenul pentru găuri a fost eliminat, deoarece agentul nu părea să învețe clar legătura dintre găurile locale și supraviețuirea pe termen lung. Cel mai util semnal a fost recompensa de contact, definită prin raportul dintre muchiile externe ale piesei care ating marginea tablei sau alte forme deja plasate și numărul total de muchii externe ale piesei.

Dacă raportul de contact este sub un prag, agentul primește penalizare. Dacă raportul depășește pragul, primește recompensă proporțională:

$$R_{\text{contact}} = \begin{cases} -\alpha \left(\frac{\tau - c}{\tau} \right)^p, & c < \tau, \\ \beta \left(\frac{c - \tau}{1 - \tau} \right)^p, & c \geq \tau, \end{cases} \quad (5)$$

unde c este raportul de contact, τ este pragul, iar α , β și p sunt hiperparametri.

Tabela 3: Familii de recompense testate în proiect.

Variantă	Idee	Observație experimentală
Recompensă pentru plasare	Bonus simplu la fiecare piesă validă	Ușor de învățat, dar nu conduce la strategie bună
Recompensă pentru linii	Bonus mare pentru linii/coloane eliminate	Necesară, dar singură nu este suficientă
Bonus de etapă	Bonus când sunt consumate toate cele trei piese	Poate încuraja supraviețuirea scurtă, nu neapărat plasarea bună
Penalizare găuri	Penalizare pentru regiuni libere mici	Semnal greu de asociat cu mutarea curentă
Contact fără prag	Bonus proporțional cu contactul piesei	Mai bun decât plasarea simplă, dar prea blând
Contact cu prag	Penalizare sub 40%, bonus peste 40%	Cea mai coerentă variantă locală
Penalizare final joc	Penalizare la blocaj	Necesară pentru a evita politici fără grijă pentru supraviețuire

6 Generatorul de piese

Generatorul de piese a fost una dintre cele mai importante componente ale proiectului. Un generator prea greu produce episoade scurte și semnal de învățare slab, iar un generator prea simplu poate duce la un agent care nu generalizează.

Au fost testate mai multe strategii:

- `random`: alege piese complet aleator;
- `solvable`: caută prin backtracking un set de trei piese care poate fi complet plasat;
- `playable`: garantează cel puțin o piesă plasabilă;
- `adaptive_playable`: folosește probabilități adaptate la spațiul liber și garantează cel puțin o opțiune jucabilă.

Strategia finală preferată este `adaptive_playable`. Aceasta păstrează mediul suficient de blând pentru învățare, dar nu elimină complet dificultatea. Piese simple apar mai des, piesele medii apar cu probabilitate moderată, iar piesele grele precum T, S, Z și diagonalele apar mai rar. În plus, piesele mari sunt mult mai probabile când tabla este goală și devin aproape imposibile când tabla este aglomerată.

Tabela 4: Comparatie între generatoarele de seturi de piese testate.

Generator	Avantaj	Limitare
<code>random</code>	Simplu și rapid	Poate genera seturi imposibile sau foarte dure
<code>solvable</code>	Garantează secvențe jucabile	Cost mare prin backtracking și joc prea artificial
<code>playable</code>	Asigură cel puțin o mutare	Nu controlează suficient dificultatea pieselor
<code>adaptive_playable</code>	Ajustează dificultatea după tablă	Necesită calibrare atentă a probabilităților

7 Workflow tehnic complet al antrenării

Această secțiune descrie fluxul efectiv al datelor în varianta principală a proiectului, adică mediul 8x8 antrenat cu PPO, *action masking*, generator adaptiv și CNN de bază. Varianta 4x4 a fost tratată ca experiment adițional pentru viteză și analiză, dar nucleul proiectului rămâne modelul 8x8, deoarece acesta corespunde mai bine jocului original și oferă spațiu suficient pentru strategii de supraviețuire. Diagrama din Figura 1 prezintă acest traseu la nivel de aplicație; în continuare sunt detaliate transformările interne, de la observație până la actualizarea modelului.

7.1 Pornirea unei rulări de antrenare

O rulare de antrenare pornește din `train.py`. Scriptul primește hiperparametri precum dimensiunea batch-ului, numărul de pași pe rollout, rata de învățare, factorul de discount, tipul CNN-ului și generatorul de piese. Pentru experimentul principal 8x8, configurația reprezentativă a fost:

- `model`: `MaskablePPO` cu `MultiInputPolicy`;
- `extractor`: `CustomCNNExtractor`;
- `mediu`: `BlockBlastEnv`;

- generator: `adaptive_playable`;
- recompensă: linii, contact, penalizare la final de joc, fără penalizare pentru găuri;
- hiperparametri importanți: $\gamma = 0.3$, rată de învățare 0.002;
- colectare și actualizare: `n_steps=1024`, `batch_size=1024`.

Scriptul creează mai multe instanțe ale mediului în paralel. Fiecare instanță simulează propriul joc, ceea ce permite colectarea mai rapidă a experiențelor. La fiecare pas, PPO cere observația curentă, masca acțiunilor valide și apoi produce o acțiune.

7.2 Observația primită de model

Pentru mediul 8x8, observația este un dicționar:

$$obs = \{\text{board}, \text{valid_actions}, \text{hand}, \text{available}\}. \quad (6)$$

Componenta `board` are forma (1, 8, 8) și reprezintă tabla curentă. O celulă are valoarea 1 dacă este ocupată și 0 dacă este liberă. Componenta `valid_actions` are forma (3, 8, 8): pentru fiecare dintre cele trei piese există o hartă binară cu pozițiile în care acea piesă poate fi plasată. Componenta `hand` are forma (3, 3, 3) și descrie cele trei piese, fiecare piesă fiind introdusă într-o matrice de maximum 3×3 . Componenta `available` are forma (3) și indică dacă fiecare slot mai poate fi folosit.

Acest design este important deoarece modelul nu vede doar tabla, ci și locurile în care fiecare piesă poate fi jucată. Fără `valid_actions`, CNN-ul ar trebui să deducă singur geometria fiecărei piese și toate suprapunerile posibile, ceea ce ar face antrenarea mult mai grea.

7.3 Calculul măștii de acțiuni

Înainte ca PPO să aleagă o acțiune, mediul calculează masca acțiunilor valide. Pentru 8x8 există 192 de acțiuni posibile:

$$a = i \cdot 64 + r \cdot 8 + c, \quad (7)$$

unde i este indexul piesei din setul curent, iar (r, c) este poziția de plasare pe tablă. Dacă piesa nu încapă sau se suprapune cu celule ocupate, acțiunea este marcată cu 0. Dacă plasarea este posibilă, acțiunea este marcată cu 1. MaskablePPO folosește această mască pentru a elimina acțiunile imposibile din distribuția politicii.

7.4 Trecerea prin CNN

Extractorul CNN de bază primește separat informația spațială și informația despre setul curent de piese. Prima ramură procesează tabla și hărțile de acțiuni valide. Acestea sunt concatenate pe dimensiunea canalelor:

$$X_{\text{spatial}} = \text{concat}(\text{board}, \text{valid_actions}). \quad (8)$$

Pentru 8x8, rezultatul are forma (4, 8, 8): un canal pentru tabla ocupată și trei canale pentru acțiunile valide ale celor trei piese. Acest tensor trece prin două straturi convoluționale:

$$(4, 8, 8) \rightarrow \text{Conv2D}(32) \rightarrow \text{ReLU} \rightarrow \text{Conv2D}(64) \rightarrow \text{ReLU}. \quad (9)$$

Primul strat convoluțional detectează configurații locale simple: vecinătăți ocupate, margini, poziții libere și zone în care o piesă poate începe. Al doilea strat combină aceste informații în caracteristici mai abstracte, precum linii aproape complete sau zone ale tablei cu multe mutări valide. Rezultatul este aplatizat într-un vector:

$$F_{\text{spatial}} \in \mathbb{R}^{64 \cdot 8 \cdot 8}. \quad (10)$$

În paralel, componenta hand este aplatizată și trecută printr-un strat liniar. Astfel, modelul primește o reprezentare compactă a formelor disponibile:

$$(3, 3, 3) \rightarrow \text{Flatten} \rightarrow \text{Linear}(64) \rightarrow \text{ReLU}. \quad (11)$$

Vectorul available este trecut printr-o altă ramură liniară:

$$(3) \rightarrow \text{Linear}(16) \rightarrow \text{ReLU}. \quad (12)$$

Cele trei rezultate sunt concatenate:

$$F = \text{concat}(F_{\text{spatial}}, F_{\text{hand}}, F_{\text{available}}). \quad (13)$$

Acest vector final trece printr-un strat liniar care produce reprezentarea comună folosită de PPO. Important este că gradientul PPO actualizează și straturile CNN, nu doar capetele finale ale politicii. CNN-ul învață treptat ce tipare spațiale sunt utile pentru deciziile care produc recompense mai bune.

Tabela 5: Fluxul tensorilor în extractorul CNN de bază pentru varianta principală 8x8.

Etapă	Intrare	Ieșire
Concatenare spațială	board (1, 8, 8) + valid_actions (3, 8, 8)	tensor (4, 8, 8)
Convoluție 1	(4, 8, 8)	(32, 8, 8)
Activare ReLU	(32, 8, 8)	(32, 8, 8)
Convoluție 2	(32, 8, 8)	(64, 8, 8)
Activare ReLU	(64, 8, 8)	(64, 8, 8)
Aplatizare spațială	(64, 8, 8)	4096 valori
Ramură piese	hand (3, 3, 3)	64 valori
Ramură disponibilitate	available (3)	16 valori
Concatenare finală	4096 + 64 + 16	4176 valori
Strat final extractor	4176 valori	features_dim = 256
Actor PPO	256 valori	scoruri pentru 192 acțiuni
Critic PPO	256 valori	valoarea scalară $V(s)$

7.5 Actorul, criticul și acțiunea aleasă

După extractorul CNN, PPO folosește două capete neuronale:

- actorul, care produce distribuția de probabilitate peste acțiuni;
- criticul, care estimează valoarea stării curente.

Actorul produce un scor pentru fiecare dintre cele 192 de acțiuni. Înainte de eșantionare, scorurile acțiunilor invalide sunt mascate. Din distribuția rămasă, PPO alege o acțiune. În timpul antrenării, alegerea este stocastică pentru a păstra explorarea. În timpul evaluării, se poate folosi alegerea deterministă, adică acțiunea cu probabilitatea cea mai mare.

Criticul produce o estimare scalară $V(s_t)$, care exprimă cât de promițătoare este starea curentă. Această estimare este folosită ulterior pentru calculul avantajului, adică pentru a decide dacă acțiunea luată a fost mai bună sau mai slabă decât se aștepta modelul.

7.6 Aplicarea acțiunii în mediu

Acțiunea aleasă este decodificată în trei componente: piesa, rândul și coloana. Mediul verifică plasarea, aplică piesa pe tablă, elimină liniile și coloanele complete, actualizează setul de piese disponibil și verifică dacă episodul s-a terminat. Apoi calculează recompensa:

$$R = R_{linie} + R_{contact} - P_{final}. \quad (14)$$

În configurația principală, recompensa pentru linii este mare, deoarece eliminarea liniilor este obiectivul direct al jocului. Recompensa de contact încurajează plasarea pieselor lipite de margini sau de forme existente, evitând plasările izolate în mijlocul tablei. Penalizarea de final de joc descurajează stările care duc rapid la blocaj.

7.7 Colectarea rollout-ului

PPO nu actualizează modelul după fiecare mutare individuală. Mai întâi colectează un rollout de experiențe din mediile paralele. Fiecare tranziție conține:

$$(s_t, a_t, r_t, s_{t+1}, done_t, \log \pi(a_t|s_t), V(s_t)). \quad (15)$$

După colectarea rollout-ului, PPO calculează avantajele. Intuitiv, avantajul răspunde la întrebarea: *a fost acțiunea aleasă mai bună decât se aștepta criticul?* Dacă da, probabilitatea acelei acțiuni trebuie crescută. Dacă nu, probabilitatea trebuie scăzută.

7.8 Actualizarea PPO

După calculul avantajelor, experiențele sunt împărțite în mini-batch-uri. Pentru fiecare mini-batch, PPO face mai multe epoci de optimizare. Funcția de pierdere are trei componente principale:

- pierderea politicii, care actualizează actorul;
- pierderea valorii, care actualizează criticul;

- bonusul de entropie, care menține explorarea.

Prin backpropagation, eroarea se propagă din actor și critic înapoi prin MLP, prin stratul de combinare și apoi în ramurile CNN și liniare. Astfel, dacă o anumită configurație spațială a dus repetat la recompense bune, filtrele convoluționale vor fi ajustate pentru a o recunoaște mai ușor. Dacă o configurație a dus la blocaje, probabilitatea acțiunilor asociate scade.

PPO a fost preferat în locul unei metode de valoare clasice deoarece spațiul de acțiuni este discret, dar puternic dependent de stare: multe acțiuni sunt invalide în fiecare moment. În combinație cu MaskablePPO, modelul poate optimiza o politică peste acțiunile posibile fără să consume antrenarea pe acțiuni imposibile. În același timp, mecanismul de clipping limitează schimbările bruște ale politicii, ceea ce este util într-un mediu în care recompensele pot varia mult de la o secvență la alta.

7.9 Monitorizarea în TensorBoard

În timpul antrenării, proiectul urmărește atât metrici de joc, cât și metrici interne PPO. Metricile de joc arată dacă agentul supraviețuiește mai mult și distruge mai multe linii. Metricile PPO arată dacă procesul de optimizare este stabil. De exemplu:

- `ep_len_mean` indică durata medie a episoadelor;
- `eval_stages/mean` indică performanța reală în evaluare;
- `approx_kl` arată cât de mult se schimbă politica la update;
- `clip_fraction` arată cât de des PPO limitează update-ul;
- entropia arată nivelul de explorare al politicii;
- `explained_variance` arată cât de bine estimează criticul valoarea stărilor.

8 Euristica de referință

Pentru a separa problema generatorului de problema modelului neuronal, a fost implementată o euristică de referință. Aceasta evaluează toate mutările valide și alege acțiunea cu scorul cel mai mare. Scorul ia în calcul:

- numărul de linii eliminate;
- contactul piesei cu marginea sau alte forme;
- numărul de acțiuni valide rămase după mutare;
- potențialul de completare a liniilor;
- dimensiunea celei mai mari regiuni libere;
- penalizarea stărilor fără mutări viitoare.

Euristica a fost importantă deoarece a demonstrat că generatorul permite jocuri lungi. În testare vizuală, euristica a depășit 100 de etape în mai multe rulări, mult peste performanța majorității modelelor inițiale de învățare prin recompensă. Această observație a indicat că limitarea principală nu este imposibilitatea mediului, ci dificultatea modelului PPO de a învăța politica potrivită doar din modelarea recompensei.

Rezultatul este asemănător cu situația întâlnită în jocuri precum șahul sau alte probleme cu spații de stări foarte mari: o euristică bine aleasă poate obține rapid rezultate puternice fără

să caute exhaustiv toate posibilitățile. O variantă cu *Monte Carlo Tree Search* ar fi fost, de asemenea, potrivită conceptual, deoarece ar putea simula mai multe continuări posibile înainte de alegerea unei mutări [12]. Totuși, pentru Block Blast acest lucru ar crește mult costul de calcul, mai ales când trebuie evaluate trei piese, multe poziții posibile și mai multe generări viitoare de piese. În proiect, euristica simplă a fost mai practică: nu garantează optimul global, dar oferă o funcție de evaluare locală foarte eficientă. Ea a avut și rol de instrument de diagnostic: dacă euristica poate juca mult, atunci generatorul și regulile jocului permit strategii bune; dacă modelul neuronal nu ajunge acolo, problema se află mai degrabă în reprezentare, antrenare sau generalizare.

9 Experimente și rezultate

9.1 Metodologia experimentală

Experimentele au fost rulate incremental. Nu a existat o singură configurație finală aleasă arbitrar, ci o succesiune de ipoteze testate: generator mai blând, penalizare pentru găuri, recompense mai rapide, discount mai mic, rată de învățare mai mare, batch-uri mai mici, arhitecturi CNN diferite și, la final, încercarea de a folosi euristica drept profesor.

O dificultate importantă a fost faptul că jocul oficial nu oferă public generatorul de piese. Dacă acesta ar fi fost disponibil, mediul de antrenare ar fi putut reproduce mai fidel distribuția reală a jocului. În lipsa lui, generatorul a trebuit aproximat și calibrat manual. Primele experimente au arătat că performanța agentului este foarte sensibilă la această componentă: un generator prea dur nu oferă suficiente șanse de învățare, iar unul prea blând produce politici care nu generalizează.

Principalele metrice urmărite au fost:

- `rollout/ep_len_mean`: lungimea medie a episodului în timpul antrenării;
- `eval_stages/mean`: numărul mediu de etape în evaluare; evaluarea era rulată periodic, de obicei la 500000 pași în experimentele lungi și mai des în sweep-urile scurte;
- `eval_stages/max`: cel mai bun episod din evaluare;
- `time/fps`: viteza de antrenare;
- `train/approx_kl`, `clip_fraction`, entropie și `explained_variance`.

9.2 Familii de experimente încercate

Tabela 6: Direcțiile majore de experimentare din proiect.

Familie	Ce s-a încercat	Concluzie
Curriculum inițial	faze simple, piese ușoare, creșterea treptată a dificultății	util pentru pornire, dar nu suficient pentru performanță stabilă

Familie	Ce s-a încercat	Concluzie
Fără găuri	eliminarea penalizării pentru regiuni libere mici	mai stabil decât penalizarea agresivă pentru găuri
Cu găuri	penalizare pentru găuri curente și găuri nou create	agentul nu asocia clar penalizarea cu decizia curentă
Seed unic	antrenare pe un singur seed fix	putea produce scoruri bune pe acel caz, dar risca supraspecializare
12 seed-uri	antrenare pe mai multe seed-uri fixe	generalizare mai bună decât seed unic, dar performanță mai mică
Finetune	continuare de la modele deja antrenate	util, dar uneori modelul devenea greu de schimbat
Sweep modele	CNN, MLP, variante mici/rapide, variante cu găuri	a arătat că arhitectura și funcția de recompensă contează simultan
Generator adaptiv	adaptive_playable, probabilități după spațiul liber	cea mai bună variantă de generator pentru training
Recompensă de contact	recompensă pentru piese lipite de margini sau alte forme	semnal local mai interpretabil decât penalizarea găurilor
Contact cu prag	penalizare sub 40%, bonus peste prag	a produs comportament mai coerent
Discount diferit	$\gamma = 0.3, 0.4, 0.5, 0.6$	$\gamma = 0.3$ a fost foarte promițător pentru decizii locale
Rată de învățare	valori mici, medii, mari și foarte mari	0.002 a fost bun; valori foarte mari au devenit instabile
Action-aware CNN	procesarea separată a fiecărei piese	conceptual bun, dar prea lent în varianta curentă
4x4	reducerea tablei pentru FPS mai mare	FPS mai bun, performanță mai slabă din cauza densității
Teacher heuristic	150000 exemple generate de euristică	aproximativ 90% acuratețe pe antrenare, dar doar 40% pe validare

9.3 Rezultate reprezentative

Tabela 7: Rezumat al unor experimente reprezentative. Valorile sunt orientative și provin din rulările TensorBoard disponibile.

Experiment	Tablă	Model	Configurație	Pași	FPS	Eval mediu	Eval max
short_g03_lr2e3	8x8	CNN base	$\gamma = 0.3$, LR=0.002	3.18M	1323	19.92	60
Gamma04_BasicContact	8x8	CNN base	$\gamma = 0.4$, LR=0.0002	16.46M	1446	17.26	31
Gamma06_BasicContact	8x8	CNN base	$\gamma = 0.6$, LR=0.0002	11.23M	1404	17.96	36
Gamma06_Threshold	8x8	CNN base	$\gamma = 0.6$, LR=0.0008	2.71M	1404	20.24	51
ActionAware	8x8	CNN action-aware	$\gamma = 0.3$, LR=0.002	0.15M	107	15.12	32
BC teacher	8x8	CNN/MLP	150000 exemple euristice	—	—	90% train	40% val
4x4_best8x8	4x4	CNN base	$\gamma = 0.3$, LR=0.002	1.00M	3258	2.54	11

Cel mai promițător model 8x8 din experimentele inițiale a fost varianta cu $\gamma = 0.3$ și rata de învățare 0.002. Aceasta a obținut o valoare maximă de evaluare de 60 în metrica urmărită și a păstrat un raport rezonabil între stabilitatea actualizărilor PPO și explorare.

Modelele cu rată de învățare foarte mare au explorat mai agresiv, dar au avut tendința de a deveni instabile. În TensorBoard, acest lucru se observă prin valori mari ale `approx_kl`, scăderea entropiei și o politică ce se blochează rapid într-un comportament suboptimal.

9.4 Observații despre experimentul 4x4

Varianta 4x4 a fost un experiment adițional. Ea reduce spațiul de acțiuni de la 192 la 48 și crește viteza de antrenare, iar în rularea inițială viteza a depășit 3000 FPS. Totuși, rezultatele au fost mai slabe decât se anticipa. Explicația este că tabla devine mult mai aglomerată: fiecare piesă ocupă un procent mai mare din spațiul total, iar o singură plasare neinspirată poate produce blocaj rapid. Din acest motiv, rezultatele principale ale proiectului sunt raportate pe 8x8, iar 4x4 este păstrat ca test de pipeline și comparație de viteză.

9.5 Grafice și capturi pentru rezultate

Capturile din TensorBoard (Figurile 2-6) constituie dovezile empirice ale evoluției antrenării, facilitând identificarea problemelor de convergență și compararea obiectivă a familiilor de modele, inclusiv analiza metricilor de joc și verificarea învățării supervizate din euristică.

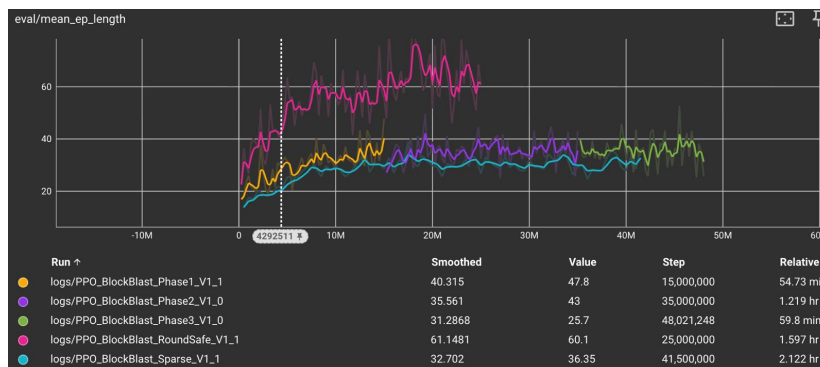


Figura 2: Primele familii de modele testate.

În Figura 2, curba albastră corespunde primului model antrenat mai mult timp într-o configurație relativ directă. Celelalte curbe reprezintă încercarea de a împărți antrenarea în trei etape cu parametri diferiți și apoi rularea în care a fost schimbat generatorul de piese. Diferența dintre aceste curbe a tras primul semnal de alarmă: generatorul influențează masiv dificultatea jocului și, implicit, ce poate învăța agentul.

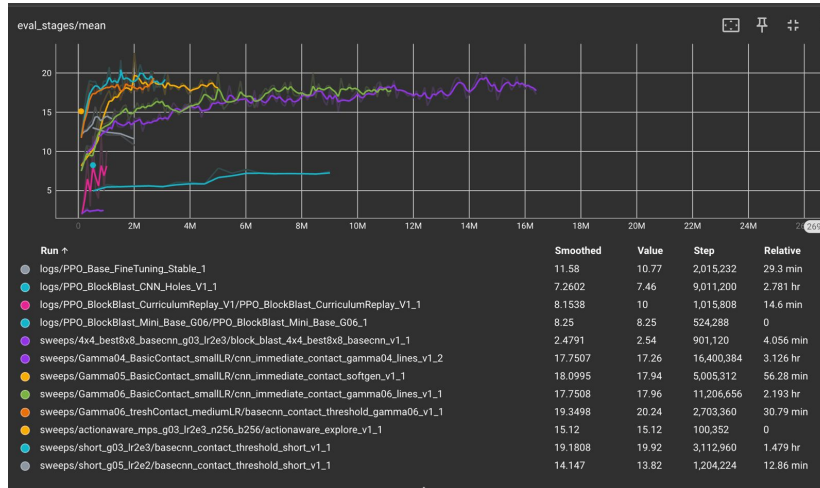


Figura 3: Evaluări periodice pentru modelele ulterioare.

Figura 3 compară modelele rulate după schimbările majore de strategie: valori diferite pentru γ , rate de învățare diferite, recompensă de contact, penalizări pentru găuri, variante alternative de CNN și experimentul 4x4. Evaluarea era făcută periodic, pe episoade separate de rollout, deci arată comportamentul modelului în jocuri de test, nu doar recompensa acumulată în timpul colectării experiențelor.

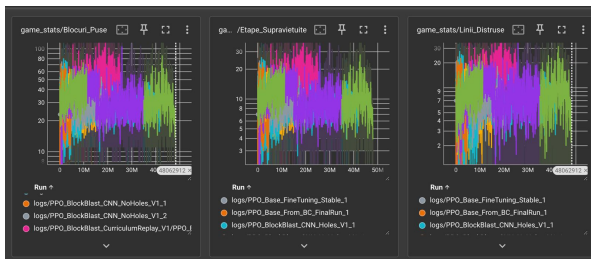


Figura 4. Metrice de joc în timpul antrenării.

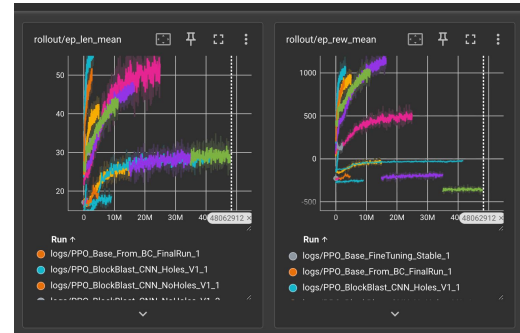


Figura 5. Rollout pentru aceleași modele.

Figurile 4 și 5 trebuie citite împreună. Prima urmărește blocurile plasate, etapele supraviețuite și liniile eliminate, iar a doua descrie datele colectate efectiv pentru actualizările PPO: lungimea episodului și recompensa medie. Variația dintre episoade este mare, lucru normal într-un joc cu piese generate procedural, dar apare totuși o ierarhie între familiile de modele. Modelele mai bune nu cresc doar recompensa totală, ci mențin simultan mai multe plasări valide, episoade mai lungi și mai multe linii eliminate.

Figura 6 arată încercarea de a folosi euristica drept profesor. Modelul a învățat aproximativ 90% din deciziile setului de antrenare, dar pe validare a rămas în jur de 40%. Diferența indică overfitting: modelul memorează bine exemplele văzute, dar nu generalizează suficient pe configurații noi ale tablei și ale pieselor.

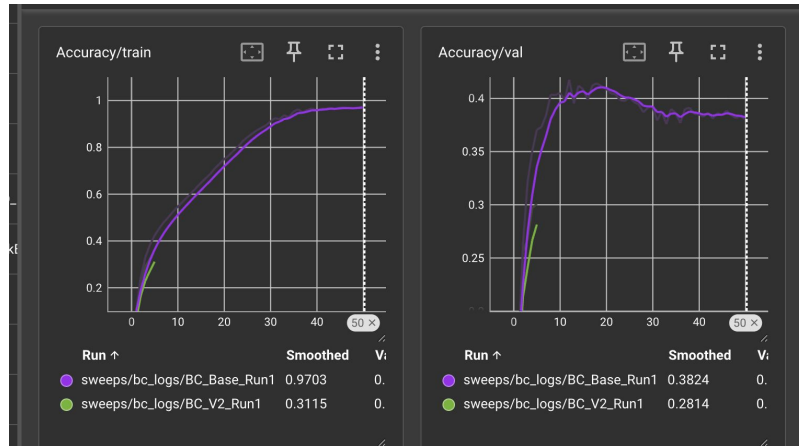


Figura 6: Învățare supervizată din euristică.

10 Testare și validare

10.1 Testare în timpul antrenării

Prima formă de testare a calității antrenării este analiza metricilor. Terminalul confirmă că rulează configurația corectă: model path, generator, penalizări activate sau dezactivate, număr de medii paralele, FPS, checkpoint-uri și device. TensorBoard arată apoi dacă agentul chiar învață sau doar acumulează recompensă accidental.

policy_gradient_loss	0.000207
value_loss	1.08e+03
game_stats/	
Blocuri_Puse	77
Etape_Supravietuite	25
Linii_Distruse	23
rollout/	
ep_len_mean	41.1
ep_rew_mean	957
step_stats/	
game_valid_actions	21.6
holes_created_cells	0
holes_current_cells	0
placement_contact_ratio	0.699
reward_contact	14.6
reward_game_over	0
reward_holes	0
reward_line	9.5
reward_stage	0
time/	
fps	1453
iterations	610
time_elapsed	3436
total_timesteps	4997120
train/	
approx_kl	1.0181524e-05
clip_fraction	0
clip_range	0.2
entropy_loss	-1.52
explained_variance	0.321
learning_rate	0.00012
loss	524
n_updates	2436
policy_gradient_loss	-0.000357
value_loss	1.15e+03
Eval @ 5000000: mean_stages=17.94, p90=26.2, max=34	

Figura 7: Afișare în terminal în timpul antrenării.

În special, s-au urmărit trei categorii:

- metrice de joc: episoade mai lungi, mai multe linii distruse, mai multe blocuri plasate;
- metrice de evaluare: performanță pe episoade separate, nu doar în rollout;
- metrice PPO: stabilitatea update-urilor și nivelul de explorare.

10.2 Testare după antrenare în `watch.py`

După terminarea sau oprirea unui model, testarea principală se face în interfața vizuală `watch.py`. O medie bună în TensorBoard nu arată mereu cum gândește agentul. Vizualizarea permite identificarea greșelilor concrete: plasări izolate, ignorarea unei linii aproape complete, blocarea unei zone mari sau alegerea unei piese care reduce drastic mutările viitoare.

Interfața conține patru moduri principale:

- **Watch AI play**: rulează modelul antrenat și permite observarea deciziilor sale pe jocuri generate automat;
- **Watch heuristic play**: rulează euristica de referință pe același tip de mediu, pentru comparație;
- **You vs AI**: permite comparația directă între jucătorul uman și agent, pe stări similare;
- **Solver / board editor**: permite construirea manuală a unei table și a unui set de piese, apoi testarea modului în care euristica sau modelul alege mutarea.

Interfața include elemente care fac testarea mai clară: afișarea tablei, desenarea pieselor disponibile, evidențierea plasării posibile, drag and drop pentru piesele mutate manual, mesaje de stare și moduri diferite de pornire. În editor, utilizatorul poate modifica tabla și piesele pentru a crea situații limită, nu doar scenarii obișnuite. Aceste moduri acoperă atât testarea cantitativă, cât și testarea calitativă. De exemplu, *Watch AI play* răspunde la întrebarea „ce face modelul singur?”, iar *Solver / board editor* răspunde la întrebarea „ce face modelul într-o situație dificilă creată intenționat?”.

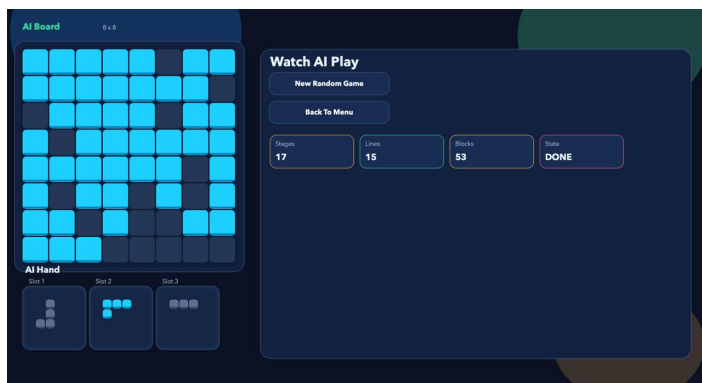


Figura 8. Interfața `watch.py` în timpul testării vizuale.

Figura 8 arată interfața folosită pentru observarea modelului după antrenare. Captura include tabla, piesele disponibile și informațiile de stare, astfel încât deciziile agentului pot fi analizate direct. Această testare este importantă deoarece un grafic bun nu explică singur dacă agentul plasează piesele compact sau dacă își blochează spațiul liber.

11 Concluzii

Din punct de vedere tehnic, cea mai importantă concluzie este că modelarea recompensei ajută, dar nu rezolvă singură problema. Penalizarea pentru găuri a fost mai puțin eficientă decât era de așteptat, deoarece agentul nu asocia suficient de clar penalizarea cu decizia care producea blocajul. În schimb, recompensa de contact a fost mai ușor de interpretat: piesele plasate lipit de margini sau alte forme tind să păstreze tabla mai compactă și mai ușor de folosit.

Cele mai coerente rezultate 8x8 au fost obținute cu CNN-ul de bază, rată de învățare 0.002 și factor de discount $\gamma = 0.3$. Această configurație a oferit un compromis bun între explorare, viteză și stabilitate. Modelele cu rată de învățare prea mică au progresat lent, iar cele cu rată

de învățare prea mare au devenit instabile. Experimentul 4x4 a arătat că reducerea spațiului de acțiuni nu garantează un joc mai ușor: tabla mai mică este mai aglomerată, iar greșelile sunt mai greu de recuperat.

Impactul proiectului este relevant pentru înțelegerea limitărilor practice ale învățării prin recompensă în jocuri puzzle. Chiar într-un joc aparent simplu, succesul depinde de reprezentarea stării, de generator, de arhitectura modelului, de semnalul de recompensă și de metoda de evaluare. Proiectul arată concret că un agent nu devine bun doar prin rularea unui număr mare de pași; are nevoie de o formulare atentă a problemei.

Direcții viitoare:

- colectarea unui dataset de mutări generate de euristică;
- pre-antrenarea politicii prin învățare prin imitare;
- rafinarea ulterioară prin PPO;
- optimizarea arhitecturii action-aware pentru viteză;
- curriculum atent calibrat de la table mici la 8x8, doar după reglarea dificultății generatorului;
- evaluare mai strictă pe seturi fixe de seed-uri.

12 Utilizarea instrumentelor LLM

În realizarea proiectului au fost folosite instrumente LLM în rol auxiliar, în special pentru elemente de interfață și mentenanță. Un exemplu important este implementarea vizuală din `watch.py`: meniuri mai clare, interacțiuni de tip drag and drop pentru piese, afișarea tablei și organizarea modurilor de testare într-o formă mai prietenoasă pentru utilizator.

Instrumentele LLM au mai fost folosite pentru îmbunătățirea mesajelor afișate în terminal în timpul antrenării, astfel încât rulările să prezinte într-o formă lizibilă date precum `model path`, `checkpoint dir`, `generator`, `FPS`, `recompense` și `metrice de joc`. De asemenea, au fost folosite punctual pentru completări de cod deja începute, de exemplu continuarea definirii formelor din `blocks.py` și gruparea lor pe categorii, pornind de la structura stabilită manual.

Bibliografie

- [1] Sutton, R. S., Barto, A. G. (2018). *Reinforcement Learning: An Introduction*. Second Edition, MIT Press.
- [2] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O. (2017). *Proximal Policy Optimization Algorithms*. arXiv:1707.06347.
- [3] Stable-Baselines3 Contributors (Accesat aprilie 2026). *SB3-Contrib Documentation: Maskable PPO*. <https://sb3-contrib.readthedocs.io/>
- [4] LeCun, Y., Bottou, L., Bengio, Y., Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), pp. 2278–2324.
- [5] Goodfellow, I., Bengio, Y., Courville, A. (2016). *Deep Learning*. MIT Press.
- [6] Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518, pp. 529–533.
- [7] Ristic, D. (Accesat aprilie 2026). *BlockBlast-Game-AI-Agent*. <https://github.com/RisticDjordje/BlockBlast-Game-AI-Agent>
- [8] Kim, Z. (Accesat aprilie 2026). *BlockBlastAI*. <https://github.com/ZacharyKim7/BlockBlastAI>
- [9] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., et al. (2019). *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. Advances in Neural Information Processing Systems, 32.
- [10] Towers, M., Terry, J. K., Kwiatkowski, A., et al. (Accesat aprilie 2026). *Gymnasium Documentation*. <https://gymnasium.farama.org/>
- [11] Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M., Dormann, N. (2021). *Stable-Baselines3: Reliable Reinforcement Learning Implementations*. Journal of Machine Learning Research, 22(268), pp. 1–8.
- [12] Kocsis, L., Szepesvári, C. (2006). Bandit based Monte-Carlo planning. *European Conference on Machine Learning*, pp. 282–293.